Individuelles Projekt im Modul Anwendungsentwicklung

Beispielprojekt: Cartogram

Jan Geller (681182)

Hochschule Worms

17. Juni 2026



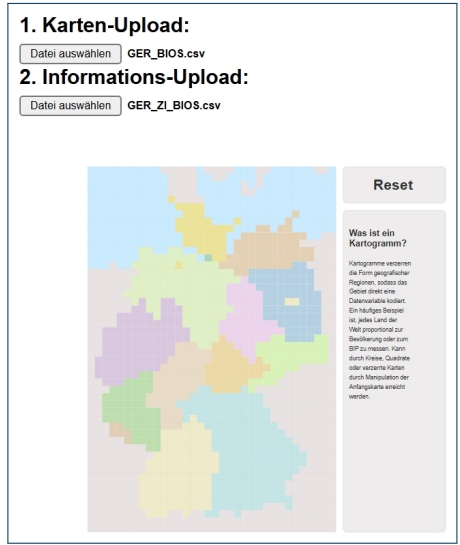
Idee Cartogram

Kartogramme verzerren die Form geografischer Regionen, sodass das Gebiet direkt eine Datenvariable kodiert. Ein häufiges Beispiel ist, jedes Land der Welt proportional zur Bevölkerung oder zum BIP zu messen. Dies kann durch Kreise, Quadrate oder verzerrte Karten durch Manipulation der Anfangskarte erreicht werden.



https://datavizproject.com/data-type/cartogram/

Geschützte Gebiete in den Bundesländern





Daten für Deutschland im CSV-Format

```
1  mmmmmmmmmmlllllmlllllmmmllmmmmmmllm
2  mmmmmmmmmmmlllllmmllmmmmlmmmmmmmmmm
3  mmmmmmmmmmmmlllllmmmmmmmlmlmmmmmmmmm
4  mmmmmmmmmmmmlllllmmmmmmllmmmmmmmmmm
5  mmmmmmmmmmmmAlllllmmmmmmmmmmmmmmmmm
6  mmmmmmmmmmmmmAAAAmmmmmmmmmmmmmmmmm
7  mmmmmmmmmmmmmAAAmmmmmmmmmmmmCmmmmm
8  mmmmmmmmmmmmmAAAAAmmmAmmmmmmCmmmmmm
9  mmmmmmmmmmmmmAAAAAAAAmmmCCmCCmmmmmm
10 mmmmmmmmmmmmmmAAAAAAmmCCCCCCCCCmmll
11 mmmmmmmmmmmmmmAAAAAAmCCCCCCCCCllllll
12 mmmmmmmDDmmDDmmAAAAmCCCCCCCCCCCllll
13 mmmmmmmDDDDDDDDmEAACCCCCCCCCCCCCllll
14 mmmlmmDDDDDDDFDDDDDCCCCCCCCCCGGGllll
15 lllllllDDDDDDDDDDDDDCCGGGGGGGGGllll
16 lllllllDDDDDDDDDDDDDDDHGGGGGGGGGllll
17 lllllllDDDDDDDDDDDDDDHHHHGGGGGGGGllll
18 lllllDDDDDDDDDDDDDDHHHHHGGGGGGGGlll
19 lllllllDMDDMMDDDDDDDDHHHHHGGGBBGGGll
20 lllllllMMDDMMMDDDDDDDHHHHGGGGGGGGll
```

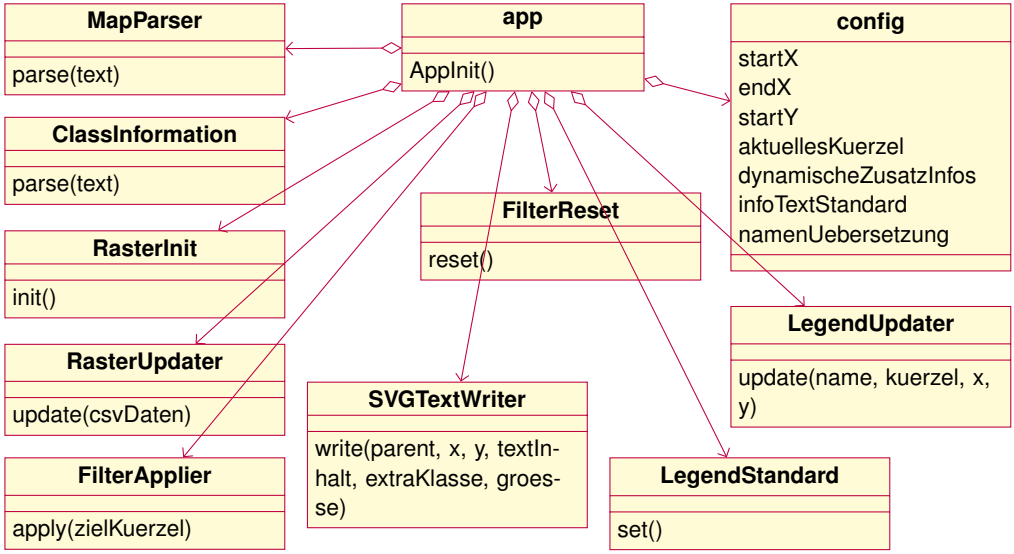
Nur ein Ausschnitt der CSV-Datei für die Generation der Karte!

Daten für Deutschland im CSV-Format

```
1  kuerzel,info
2  A,Einwohner: 2.9 Mio.   Hauptstadt: Kiel              Größe: 15.800 km       Biosphärenreservat:
         4.459 km    Naturschutzgebiete: 8.4 %       Nationalparks: 28.0 %
3  B,Einwohner: 3.7 Mio.   Hauptstadt: Berlin            Größe: 891 km          Biosphärenreservat: /
              Naturschutzgebiete: 1.5 %       Nationalparks: /
4  C,Einwohner: 1.6 Mio.   Hauptstadt: Schwerin           Größe: 23.211 km       Biosphärenreservat: 964
         km      Naturschutzgebiete: 3.5 %       Nationalparks: 4.9 %
5  D,Einwohner: 8.0 Mio.   Hauptstadt: Hannover           Größe: 47.710 km       Biosphärenreservat:
         4.275 km    Naturschutzgebiete: 3.1 %       Nationalparks: 7.4 %
6  E,Einwohner: 1.9 Mio.   Hauptstadt: Hansestadt Hamburg  Größe: 755 km         Biosphärenreservat: 117
         km      Naturschutzgebiete: 8.0 %       Nationalparks: 1.8 %
7  F,Einwohner: 0.7 Mio.   Hauptstadt: Hansestadt Bremen   Größe: 419 km          Biosphärenreservat: /
              Naturschutzgebiete: 8.8 %       Nationalparks: /
8  G,Einwohner: 2.6 Mio.   Hauptstadt: Potsdam            Größe: 29.654 km       Biosphärenreservat:
         2.301 km    Naturschutzgebiete: 8.3 %       Nationalparks: 0.35 %
9  H,Einwohner: 2.2 Mio.   Hauptstadt: Magdeburg          Größe: 20.000 km       Biosphärenreservat:
         1.421 km    Naturschutzgebiete: 5.0 %       Nationalparks: 0.7 %
10 I,Einwohner: 4.1 Mio.   Hauptstadt: Dresden           Größe: 18.450 km       Biosphärenreservat: 301
         km      Naturschutzgebiete: 3.9 %       Nationalparks: 0.5 %
11 J,Einwohner: 2.1 Mio.   Hauptstadt: Erfurt            Größe: 16.202 km       Biosphärenreservat: 915
         km      Naturschutzgebiete: 2.7 %       Nationalparks: 0.31 %
12 K,Einwohner: 6.3 Mio.   Hauptstadt: Wiesbaden         Größe: 21.115 km       Biosphärenreservat: 648
         km      Naturschutzgebiete: 1.7 %       Nationalparks: 0.27 %
```

Nur ein Ausschnitt der CSV-Datei für Zusatzinformationen!

Klassendiagramm Cartogram





Kleine algorithmische Herausforderung

- Das Programm kann beim Drüberfahren mit der Maus die Prozente anzeigen.
- Das Programm kann über alle Bundesländer den geschützten Bereich berechnen.



Kleine UI-Herausforderung

- Beim Drüber gehen mit der Maus, werden andere Bundesländer ausgegraut.
- Das Programm muss in der Lage sein, für andere CSV-Dateien eine saubere Darstellungsweise zu erzeugen.

Meine Funktionen

- RasterInit
- RasterUpdater
- MapParser
- Zusammenspiel der ersten Drei
- AppInit
- LegendUpdater
- ClassInformation



Der RasterInit I

```
1  class RasterInit {
2      init() {
3          let container = document.getElementById('raster-container');
4          container.innerHTML = '';
5          for (let y = startY; y <= endY; y++) {
6              for (let x = startX; x <= endX; x++) {
7                  let rect = document.createElementNS("http://www.w3.org/2000/svg", "rect");
8                  rect.setAttribute("x", x);
9                  rect.setAttribute("y", y);
10                 rect.setAttribute("width", "1");
11                 rect.setAttribute("height", "1");
12                 rect.setAttribute("id", "cell-" + x + "-" + y);
13                 rect.classList.add("raster-kaestchen");
14
15                 rect.addEventListener('click', function () {
16                     new FilterReset().reset();
17                     new LegendUpdater().update("Leeres Kaestchen", "-", x, y);
18                 });
19                 container.appendChild(rect);
20             }
21         }
22     }
23 }
```



Der RasterInit II

Hauptaufgabe:
Die Klasse `RasterInit` baut automatisch ein **interaktives Gitternetz (Raster)** aus mathematischen SVG-Quadraten auf dem Bildschirm auf.

Der RasterInit III

Schritt 1 & 2 - Vorbereitung und Gitter-Erzeugung

- **Aufräumen (`container.innerHTML = '';`):**
  - Bevor neu gezeichnet wird, wird alter Inhalt gelöscht.
  - Verhindert doppelte Gitter-Überlagerungen.
- **Die Schleifen (`for`-Schleifen):**
  - Das Programm läuft zeilenweise durch (Y-Achse) und spaltenweise (X-Achse).
  - An jedem Punkt wird ein SVG-Quadrat (`<rect>`) erzeugt.
- **Eigenschaften des Quadrats:**
  - **Größe:** Immer exakt 1×1 Einheit.
  - **ID:** Jedes Kästchen bekommt einen eindeutigen Namen anhand seiner Koordinaten (z. B. `cell-5-10`).

Der RasterInit IV

Schritt 3 - Interaktivität (Das Klick-Event)

Jedes einzelne Kästchen wird mit einem Event-Listener (`addEventListener`) ausgestattet. Sobald der Nutzer auf ein Kästchen klickt, passieren zwei Dinge:

1. **Filter zurücksetzen (`FilterReset`):**
   - Eventuell aktive Filter auf der Benutzeroberfläche werden aufgehoben.
2. **Legende aktualisieren (`LegendUpdater`):**
   - Das Programm gibt der Ansicht Bescheid: *„An Koordinate X/Y wurde ein leeres Kästchen angeklickt“*.

Der RasterUpdater I

```
class RasterUpdater {
    update(csvDaten) {
        new LegendStandard().set();
        aktuellesKuerzel = null;
        for (let y = startY; y <= endY; y++) {
            for (let x = startX; x <= endX; x++) {
                let rect = document.getElementById("cell-" + x + "-" + y);
                if (!rect) continue;
                rect.className.baseVal = "raster-kaestchen";
                let kuerzel = csvDaten[x + ',' + y];
                if (kuerzel) {
                    rect.classList.add(kuerzel);
                    rect.onclick = function () {
                        if (aktuellesKuerzel === kuerzel) {
                            new FilterReset().reset();
                            new LegendStandard().set();
                        } else {
                            aktuellesKuerzel = kuerzel;
                            new FilterApplier().apply(kuerzel);
                            let name = namenUebersetzung[kuerzel] || "Unbekannt";
                            new LegendUpdater().update(name, kuerzel, x, y);
                        };
                } else {
                    rect.onclick = function () {
                        new FilterReset().reset();
                        new LegendUpdater().update("Leeres Kaestchen", "-", x, y);
                    };
}}}}}
```

Der RasterUpdater II

Hauptaufgabe:
Der `RasterUpdater` verbindet die Optik (`RasterInit`) mit den Daten (`MapParser`). Er färbt die Karte ein und steuert die interaktiven Filter.

Der Ablauf in drei Schritten:

1. **Reset:** Karte und Legende werden auf den Startzustand zurückgesetzt.
2. **Einfärben:** Kästchen erhalten ihre optischen Klassen basierend auf den Daten.
3. **Intelligenz:** Aktivierung einer dynamischen Filter-Logik bei Klick.

Der RasterUpdater III

Schritt 1 & 2 - Bereinigung und visuelle Zuordnung

- **Schnittstelle zu den Daten:**
  - Das Programm prüft über `csvDaten[x + ',' + y]`, ob für das aktuelle Kästchen ein Datenwert (Kürzel) existiert.
- **Visuelles Update via CSS:**
  - Mit `rect.className.baseVal` werden alte Stile gelöscht.
  - Existiert ein Kürzel, wird es mit `classList.add(kuerzel)` als Klasse hinzugefügt. Das CSS steuert daraufhin die Farbe (z. B. Grau für Land, Blau für Wasser).

Der RasterUpdater IV

Schritt 3 - Die Klick- und Filter-Logik (Toggle-Prinzip)

Beim Klick auf ein farbiges Kästchen verhält sich die Anwendung wie ein **Lichtschalter**:

Zustand A: Filter ist noch NICHT aktiv

- Das Kürzel wird als `aktuellesKuerzel` gespeichert.
- Der `FilterApplier` wird gestartet (z. B. alles ausblenden, außer Wasser).
- Die Legende zeigt Details zum ausgewählten Element an.

Zustand B: Filter IST BEREITS aktiv (Erneuter Klick)

- `FilterReset` wird ausgeführt → Die Filterung wird aufgehoben, alle Kästchen sind wieder sichtbar.



Der MapParser I

```
1  class MapParser {
2      parse(text) {
3          let zeilen = text.split('\n');
4          let datenMap = {};
5          let y = 0;
6          for (let i = 0; i < zeilen.length; i++) {
7              let zeile = zeilen[i].trim();
8              if (!zeile || zeile.includes('matrix_zeile') || zeile.startsWith('y,')) {
9                  continue;
10             }
11             zeile = zeile.replace(/\s+/g, '');
12             for (let x = 0; x < zeile.length; x++) {
13                 datenMap[x + ',' + y] = zeile[x];
14             }
15             y++;
16         }
17         return datenMap;
18     }
19 }
```



Der MapParser II

Hauptaufgabe:
Die Klasse `MapParser` liest einen unstrukturierten Textblock (z. B. eine Kartendatei) ein und übersetzt ihn in ein maschinenlesbares **X/Y-Koordinatensystem**.

Das Prinzip dahinter:

- Text wird in Zeilen zerlegt.
- Unwichtige Zeilen (Metadaten/Header) werden ignoriert.
- Jedes verbleibende Zeichen wird exakt einer Position zugewiesen.

Der MapParser III

Schritt 1 - Text filtern und bereinigen

- **Zeilenaufteilung (`split('\n')`):**
  - Der Fließtext wird beim Zeilenumbruch getrennt, damit Zeile für Zeile gearbeitet werden kann.
- **Die Filter-Bedingung (`continue`):**
  - Leere Zeilen oder Zeilen mit den Inhalten `matrix_zeile` oder `y`, werden übersprungen.
  - Das schützt das System vor Fehlern durch Beschriftungen oder Metadaten.
- **Whitespace-Killer (`replace(/\s+/g, '')`):**
  - Alle verbleibenden Leerzeichen innerhalb einer Zeile werden gelöscht, um eine lückenlose Datenreihe zu garantieren.

Der MapParser IV

Schritt 2 - Das Mapping (Koordinaten-Vergabe)

Nachdem eine Zeile bereinigt wurde, läuft das Programm Zeichen für Zeichen durch:

- **Die Y-Achse:**
  - Wird nur hochgezählt (`y++`), wenn eine Zeile erfolgreich verarbeitet wurde. Sie repräsentiert die Zeilennummer.
- **Die X-Achse:**
  - Die innere Schleife geht die Zeichen von links nach rechts durch (`x`).
- **Das Ergebnis im Speicher (`datenMap`):**
  - Ein Eintrag sieht danach so aus: `"X,Y: SZeichen"`.
  - Beispiel: `datenMap["2,0"] = Ä"` bedeutet: In der ersten Zeile ($Y = 0$) steht an dritter Stelle ($X = 2$) der Wert Ä".

Das Zusammenspiel der ersten drei Klassen

Wie die drei Komponenten ineinandergreifen:

- **1. RasterInit (Die Oberfläche):**
  - Erstellt das leere, mathematische Gitter auf dem Bildschirm.
  - Bereitet die Struktur für die späteren Klick-Ereignisse vor.
- **2. MapParser (Die Datenbasis):**
  - Liest die rohen Textdateien ein und filtert störende Metadaten heraus.
  - Übersetzt die Zeichen in ein logisches X/Y-Koordinatensystem.
- **3. RasterUpdater (Die Fusion):**
  - Bringt die visuelle Oberfläche und die logischen Daten zusammen.
  - Färbt die Kästchen ein und steuert die interaktive Filter-Logik.

Gemeinsam bilden sie ein vollständiges, interaktives Kartensystem!

Der AppInit I

```
class AppInit {init() {document.addEventListener("DOMContentLoaded", function () {
            new RasterInit().init();
            new LegendStandard().set();
            document.getElementById('reset-button').addEventListener('click', function () {
                new FilterReset().reset();
                new LegendStandard().set();});});
        document.getElementById('csvUpload').addEventListener('change', function (event) {
            let datei = event.target.files[0];
            if (!datei) return;
            let reader = new FileReader();
            reader.onload = function (e) {
                let datenMap = new MapParser().parse(e.target.result);
                new RasterUpdater().update(datenMap);};
            reader.readAsText(datei);});
        document.getElementById('infoCsvUpload').addEventListener('change', function (event) {
            let datei = event.target.files[0];
            if (!datei) return;
            let reader = new FileReader();
            reader.onload = function (e) {
                let infoObj = new ClassInformation();
                dynamischeZusatzInfos = infoObj.parse(e.target.result);
                if (aktuellesKuerzel) {
                    let name = namenUebersetzung[aktuellesKuerzel] || "Unbekannt";
                    new LegendUpdater().update(name, aktuellesKuerzel, "?", "?");}};
            reader.readAsText(datei);});}}
```

Der AppInit II

Hauptaufgabe:
Die Klasse `AppInit` verwaltet die Event-Listener der Anwendung. Sie verknüpft die Aktionen des Nutzers auf der Oberfläche direkt mit der Programmlogik im Hintergrund.

Die drei zentralen Zuständigkeitsbereiche:

- **Seitenstart & Reset:** Initialisiert das leere System und steuert den Reset-Button.
- **Karten-Upload:** Verarbeitet die hochgeladene Text- oder CSV-Karte.
- **Zusatzinfo-Upload:** Lädt dynamische Zusatzdaten für die Legende nach.

Der AppInit III

Seitenstart und Karten-Upload

- **Schnittstelle beim Laden (`DOMContentLoaded`):**
  - Sobald die Webseite bereit ist, wird `RasterInit` gestartet.
  - Ein Klick auf den Reset-Button setzt alle aktiven Filter sofort wieder zurück.
- **Automatisierter Karten-Import (`csvUpload`):**
  - Ein `FileReader` liest die hochgeladene Datei asynchron als Text ein.
  - Die Daten werden direkt an den `MapParser` übergeben.
  - Das Ergebnis wird sofort genutzt, um das Raster via `RasterUpdater` einzufärben.

Der LegendUpdater I

```
class LegendUpdater {
    update(name, kuerzel, x, y) {
        let textGroup = document.getElementById('legende-text-group');
        textGroup.innerHTML = '';
        let textX = 35.8;
        let startYText = 9.5;
        let zeilenAbstand = 1.5;
        let maxZeichen = 30;
        let writer = new SvgTextWriter();
        writer.write(textGroup, textX, startYText, "INFO:", "legende-titel", 1.3);
        let schriftGroesse = name.length > 15 ? 0.9 : 1.1;
        writer.write(textGroup, textX, startYText + 2.5, name, "legende-titel", schriftGroesse);
        let zusatzInfo = dynamischeZusatzInfos[kuerzel];
        if (zusatzInfo) {
            startYText += 5.0;
            let woerter = zusatzInfo.split(' ');
            let aktuelleZeile = "";
            for (let i = 0; i < woerter.length; i++) {
                let testZeile = aktuelleZeile + woerter[i] + " ";
                if (testZeile.length > maxZeichen && aktuelleZeile.length > 0) {
                    writer.write(textGroup, textX, startYText, aktuelleZeile.trim(), "", 0.9);
                    startYText += zeilenAbstand;
                    aktuelleZeile = woerter[i] + " ";
                } else {
                    aktuelleZeile = testZeile;}}
            if (aktuelleZeile.length > 0) {
                writer.write(textGroup, textX, startYText, aktuelleZeile.trim(), "", 0.9);
            }}}}
```

Der LegendUpdater II

Hauptaufgabe:
Der `LegendUpdater` gibt Objektnamen und detaillierte Beschreibungen im Info-Panel aus.

Die zwei wichtigsten Kernfunktionen:

- **Dynamische Schriftgröße:** Verhindert das Herausragen langer Namen aus dem Layout.
- **Algorithmus für Zeilenumbruch:** Berechnet die optimale Textaufteilung basierend auf der Zeichenanzahl.

Der LegendUpdater III

Layout und intelligente Überschrift

- **Vorbereitung & Bereinigung:**
  - Mit `textGroup.innerHTML = ''` wird der alte Inhalt restlos gelöscht.
  - Das Objekt `SvgTextWriter` wird als Werkzeug für die Textplatzierung instanziiert.
- **Dynamische Skalierung (`name.length > 15`):**
  - Das Programm misst die Länge des Objektnamens.
  - Kurze Namen erhalten die Schriftgröße `1.1`.
  - Namen mit mehr als 15 Zeichen werden auf `0.9` herabgestuft, um das UI-Design zu schützen.

Der LegendUpdater IV

Der maßgeschneiderte Zeilenumbruch (Word Wrap)
Da SVG-Texte standardmäßig nur in einer einzigen, endlosen Zeile geschrieben werden, nutzt die Klasse eine mathematische Schleife:

- **Text-Splitting:**
  - Die Zusatzbeschreibung wird bei den Leerzeichen in einzelne Wörter zerlegt (`split(' ')`).
- **Zeichen-Prüfung (`maxZeichen = 30`):**
  - Die Schleife fügt Test-Wörter zusammen. Sobald eine Zeile länger als 30 Zeichen werden würde, wird die aktuelle Zeile gedruckt.
  - Die Y-Koordinate wird um den `zeilenAbstand` (1.5) nach unten verschoben, und das nächste Wort beginnt eine neue Zeile.
- **Resteverarbeitung:**
  - Nach der Schleife wird das letzte, verbleibende Textfragment automatisch ausgegeben.

Die ClassInformation I

```
class ClassInformation {
    parse(text) {
        let zeilen = text.trim().split('\n');
        let infoMap = {};
        for (let i = 0; i < zeilen.length; i++) {
            let zeile = zeilen[i].trim();
            if (!zeile || zeile.toLowerCase().includes('kuerzel,info')) {
                continue;
            }
            let kommaIndex = zeile.indexOf(',');
            if (kommaIndex !== -1) {
                let kuerzel = zeile.substring(0, kommaIndex).trim();
                let infoText = zeile.substring(kommaIndex + 1).trim();
                infoMap[kuerzel] = infoText;
            }
        }
        return infoMap;
    }
}
```

Die ClassInformation II

Hauptaufgabe:
Die Klasse `ClassInformation` verarbeitet Textdateien, die zusätzliche Erklärungen zu den Karten-Kürzeln enthalten.

Der Unterschied zum MapParser:

- Der `MapParser` liest die **geografische Struktur** der Karte (Koordinaten).
- `ClassInformation` liest die **Hintergrund-Informationen** zu den einzelnen Elementen (Metadaten).

Die ClassInformation III

Robuste Texttrennung im Detail

Das Programm analysiert die Datei Zeile für Zeile und nutzt dabei ein besonders fehlerresistentes Verfahren:

- **Filterung der Kopfzeile:**
  - Tabellenüberschriften wie `kuerzel,info` werden automatisch erkannt und übersprungen.
- **Das „Erste-Komma-Prinzip“ (`indexOf`):**
  - Es wird ausschließlich nach dem *ersten* Komma einer Zeile gesucht.
  - Alles links davon wird zum **Kürzel** (Key).
  - Alles rechts davon wird zum **Informationstext** (Value).
- **Vorteil der Methodik:**
  - Erlaubt es, dass im eigentlichen Informationstext reguläre Kommas verwendet werden dürfen, ohne dass der Parser abstürzt oder Daten falsch trennt.

Die ClassInformation IV

Datenfluss und Nutzen

- **Speicherung im System (`infoMap`):**
  - Die Daten werden als Schlüssel-Wert-Paare exportiert (z. B. `{"W: "Waldfläche mit dichtem Bestand"}`).
- **Zusammenhang mit anderen Klassen:**
  - `AppInit` fängt den Datei-Upload ab und jagt ihn durch diesen Parser.
  - Das Ergebnis wird global als `dynamischeZusatzInfos` bereitgestellt.
  - `LegendUpdater` greift bei jedem Klick auf diese Daten zu, um das Text-Panel auf dem Bildschirm zu füllen.